

API операционных систем

Процессы

Подготовил
Старший преподаватель
МИЭМ НИУ ВШЭ
Морозов Владимир Игоревич

Общая информация

Основные положения

- Компьютерная программа сама по себе — лишь пассивная последовательность инструкций. В то время как **процесс** — непосредственное выполнение этих инструкций.
- Т.е. **процесс** — некоторая сущность в ОС, представляющая собой активное выполнение инструкций той или иной программы
- Одну программу может одновременно выполнять несколько процессов (как, например, одну песню могут исполнять несколько певцов)
- Но один процесс в один момент времени может выполнять только одну программу (так же, как и певец не может петь две песни одновременно)

Основные положения

- Чтобы выполнить любое действие в ОС, необходим процесс, который будет это делать
- Чтобы работать с системой, пользователь запускает соответствующие конкретной задаче процессы, например `bash` или `cmd` и даёт им команды
- С целью делегирования задач одни процессы могут запускать другие, а те – ещё и т.д.
- Процесс, запустивший другой процесс, называется **родительским** для него, а запущенный процесс по отношению к своему родительскому называется **дочерним**
- Один родительский процесс может иметь несколько дочерних, но не наоборот

Идентификация и ввод-вывод

- Каждый процесс имеет свой уникальный идентификатор в пределах ОС
- Такой идентификатор называется PID (process ID)
- Процессы также могут иметь PPID (parent process ID) – идентификатор своего родительского процесса
- Процесс может принимать аргументы на вход (даже если он запущен не с командной строки) и всегда возвращает код завершения на выходе
- Дополнительно процесс может осуществлять ввод-вывод через стандартные потоки (как правило, консоль), файлы или через другие средства межпроцессного взаимодействия

Linux

Системные вызовы для работы с процессами

- `fork` – создаёт **полную копию** текущего процесса за тем лишь исключением, что в родительский процесс возвращается `pid` нового процесса, а в дочерний – 0
- `execXX` – заменяет **текущий** процесс на новый, исполняемый файл которого передан в качестве аргумента
- `_exit` – выполняет завершение процесса с кодом, переданным в качестве аргумента
- `waitpid` – выполняет ожидание завершения процесса и предоставляет код его завершения в качестве выходных данных

Создание нового процесса

- Создание нового процесса, как правило, производится комбинацией вызовов `fork` и `exec`
- С помощью `fork` создаётся копия текущего процесса (но уже с новым `pid`), после чего она тут же (или после некоторой предварительной подготовки) заменяется на абсолютно новый процесс, имя исполняемого файла которого передаётся вызову `exec`
- Таким образом, после вызова `fork` один и тот же код начинает выполняться уже двумя процессами параллельно. Чтобы отличить, в каком процессе мы находимся, нужно проверить возвращаемое значение `fork`. Для родительского процесса оно будет равно `pid` порождённого процесса, для дочернего – 0.

Создание нового процесса

- В случае ошибки оба вызова возвращают `-1`
- Для использования данных вызовов необходимо подключить заголовочный файл `unistd.h`
- В примере на следующем слайде с помощью вызова `fork` создаётся новый процесс. В новом процессе (`fork` вернёт `0`) будет запущен новый исполняемый файл. В родительском процессе программа завершится с успешным кодом `0`. В случае ошибки создания процесса будет выведено соответствующее сообщение.
- Обратите внимание, что `default`-часть инструкции `switch` реализована не вполне корректно в данном и следующем примерах

```
pid_t res = fork();
switch(res)
{
    case 0:
    {
        execve("./subproc", NULL, NULL);
    }
    case -1:
    {
        perror("Error after calling fork");
        exit(EXIT_FAILURE);
    }
    default:
    {
        exit(EXIT_SUCCESS);
    }
}
```

Передача аргументов и среды

- В качестве второго аргумента `execve` принимает массив строк в стиле Си, которые будут переданы в качестве аргументов командной строки дочернему процессу (в аргумент `argv`). **Последним элементом массива должен быть `NULL`.**
- Если передать `NULL` вместо всего массива (как это сделано в предыдущем примере), дочерний процесс получит 0 аргументов (не будет передано даже имя исполняемого файла).
- Третий аргумент используется для передачи переменных среды, которые не будут рассмотрены в рамках данного занятия
- Вызов `execve` имеет ряд аналогов из библиотеки Си, позволяющих игнорировать некоторые его аргументы или передавать их в другом виде

Передача аргументов и среды

В примере на следующем слайде родительский процесс передаёт три аргумента командной строки дочернему. Дочерний, в свою очередь, печатает свой pid и переданные ему аргументы. Результат работы программы будет выглядеть следующим образом:

I'm process with pid 4135

argv[0] = subproc

argv[1] = Hello

argv[2] = world!

```
pid_t res = fork();
switch(res)
{
    case 0:
    {
        char * args[4] = {"subproc", "Hello", "world!", NULL};
        execve("./subproc", args, NULL);
    }
    case -1:
    {
        perror("Error after calling fork");
        exit(EXIT_FAILURE);
    }
    default:
    {
        exit(EXIT_SUCCESS);
    }
}
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char * argv[])
{
    printf("I'm process with pid %d\n", getpid());

    for (int i = 0; i < argc; ++i)
    {
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    exit(EXIT_SUCCESS);
}
```

Завершение процесса

- Для завершения процесса используется системный вызов `_exit` или его библиотечный синоним `exit`. Второй вариант более предпочтителен, так как выполняет некоторую подготовку перед завершением.
- В качестве аргумента `exit` (как и `_exit`) принимает код завершения – числовое значение, передаваемое родительскому процессу и означающее, как правило, факт успешности или провала выполнения программы
- Удобнее всего использовать две предопределённых константы в качестве возвращаемых значений для успешного и ошибочного завершения работы процесса соответственно: `EXIT_SUCCESS` и `EXIT_FAILURE`

Завершение процесса

- Необходимо помнить, что, несмотря на то, что `exit` принимает аргумент типа `int`, на деле в качестве возвращаемого кода родителю будет передан только один байт этого числа
- В связи с этим не следует использовать код возврата для передачи какой-либо полезной информации родительскому процессу (или по крайней мере следует делать это с осторожностью)

Ожидание завершения дочернего процесса

- Чтобы дочерний процесс корректно завершил работу и освободил системные ресурсы, кто-то должен дождаться его завершения
- Как правило, это делает родительский процесс с помощью вызова `wait`, но есть и более плохие сценарии
- Если родительский процесс завершается, не дождавшись дочернего (т.е. не вызвав `wait` для него), родителем “сироты” становится процесс с `pid 1` (`systemd` или `init`)
- Если же родительский процесс не завершился, но так и не вызвал `wait` для дочернего, дочерний превращается в “зомби”. Как классического зомби, его нельзя убить (вызовом `kill`). Таким образом, “зомби” будет тратить некоторую (хотя и небольшую) часть системных ресурсов до тех пор, пока родительский процесс либо не завершится, либо не вызовет для него `wait`.

Ожидание завершения дочернего процесса

- Дождаться завершения дочернего процесса родительский может с помощью системного вызова `wait`
- Данный вызов принимает в качестве аргумента указатель на переменную, в которую будет помещён код возврата потомка, а возвращает `pid` завершённого процесса
- Если родительский процесс запустил несколько дочерних, `wait` дождётся только того из них, кто завершится первым. Для остальных его нужно будет вызвать повторно.
- `wait` имеет вариацию `waitpid`, которая позволяет явно указать, завершение какого процесса ожидается. Данный вызов работает аналогично `wait`, только первым аргументом принимает `pid` нужного потомка.

Ожидание завершения дочернего процесса

- Для получения корректного кода возврата из дочернего процесса следует использовать макрос `WEXITSTATUS`
- В примере на следующем слайде показано, как должна выглядеть корректно реализованная часть инструкции `switch` из предыдущего примера, чтобы дочерний процесс был нормально завершён
- Для использования вызова `wait` необходимо подключить заголовочный файл `wait.h`

default:

```
{  
  
    int code = 0;  
  
    pid_t child = wait(&code);  
  
    if (child == -1)  
    {  
  
        perror("Error after calling wait");  
  
        exit(EXIT_FAILURE);  
  
    }  
  
    printf("Process with pid %d exited with code %d\n", child, WEXITSTATUS(code));  
  
    exit(EXIT_SUCCESS);  
  
}
```

Windows

Функции для работы с процессами

- **CreateProcess** – создаёт процесс на основе исполняемого файла, передаваемого в одном из аргументов
- **ExitProcess** – завершает текущий процесс, принимая код возврата в качестве аргумента
- **WaitForSingleObject** (**MultipleObjects**) – выполняет ожидание перевода объекта (или нескольких объектов) в сигнальное состояние (процесс переходит в такое состояние при завершении)
- **GetExitCodeProcess** – получает код возврата процесса по его дескриптору, если процесс был завершён

Создание нового процесса

- Функция **CreateProcess** является ярким представителем гибких и многофункциональных элементов WinAPI и принимает 10 аргументов. В рамках данного занятия будут рассмотрены только некоторые из них.
- Функция возвращает **TRUE** в случае успешного выполнения и **FALSE** в случае ошибки
- Выходными данными функции являются PID и дескриптор созданного процесса, они помещаются в специальную структуру, передаваемую последним аргументом

Аргументы функции CreateProcess

- **lpApplicationName** – имя исполняемого файла (путь к нему) с расширением
- **lpCommandLine** – **одна строка**, содержащая все аргументы командной строки для создаваемого процесса, разделённые пробелом
- **lpStartupInfo** – указатель на структуру, содержащую информацию для запуска процесса. Должна быть заранее инициализирована функцией **GetStartupInfo**.
- **lpProcessInformation** – указатель на неинициализированную структуру, в которую после запуска нового процесса будут помещены его PID и дескриптор (а также ID и дескриптор главного потока созданного процесса)

Пример

- В примере на следующем слайде показан наиболее простой способ создания дочернего процесса
- Большинство аргументов функции `CreateProcess` в данном случае оставлены по умолчанию (как правило, это значения 0 или `NULL`)
- Результатом успешной работы программы из следующего примера будет распечатанный в консоли `pid` дочернего процесса
- Обратите внимание, что по завершении работы необходимо закрыть дескрипторы как процесса, так и его основного потока

```
PROCESS_INFORMATION pi;

STARTUPINFO si;

GetStartupInfo(&si);

BOOL res = CreateProcess("SubProcess.exe",

    NULL,

    NULL,

    NULL,

    TRUE,

    0,

    NULL,

    NULL,

    &si,

    &pi);

if (!res)

{

    printf("Error %x\n", GetLastError());

}

printf("%d\n", pi.dwProcessId);
```

Передача аргументов

- Чтобы передать дочернему процессу аргументы командной строки, необходимо передать их в виде одной Си-строки, представляющей собой последовательную запись всех аргументов, разделённых пробелами

```
BOOL res = CreateProcess(  
    "SubProcess.exe",  
    "SubProcess.exe arg1 arg2 arg3" ,  
    NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi);
```

- При этом, если в списке аргументов первым идёт название исполняемого файла, первый аргумент функции `CreateProcess` можно установить в `NULL`

Структура `PROCESS_INFORMATION`

- Выходные данные функции `CreateProcess` помещаются в заранее созданную структуру `PROCESS_INFORMATION`, имеющую следующие поля:
 - `hProcess` – дескриптор дочернего процесса
 - `hThread` – дескриптор основного потока дочернего процесса
 - `dwProcessId` – PID дочернего процесса
 - `dwThreadId` – ID основного потока дочернего процесса
- Эти поля заполняются во время работы функции и могут быть использованы для обращения к дочернему процессу или его основному потоку

Завершение процесса

- Для завершения процесса используется функция `ExitProcess`
- В качестве аргумента `ExitProcess` принимает код завершения, представленный беззнаковым целым
- В Windows нет ограничений на количество используемых бит в коде завершения, так что большие значения тоже допустимы
- В Windows нет специальных констант для кодов корректного и ошибочного завершения процессов, для этих целей функции `ExitProcess` можно передавать значения 0 и 1 соответственно

Ожидание завершения дочернего процесса

- В Windows для родителя нет необходимости дожидаться завершения дочернего процесса, т.к. “родственные связи” между процессами в данной ОС более слабы
- Однако, если завершения процесса всё же необходимо дождаться (например, для получения результатов его работы), для этого используется универсальная функция `WaitForSingleObject`
- В качестве аргументов функция принимает дескриптор дочернего процесса и таймаут ожидания. Таймаут может принимать следующие значения:
 - 0 – чтобы просто проверить, завершён ли процесс, если нет, функция сразу возвращает управление с соответствующим кодом
 - $0 < x < \text{INFINITE}$ – чтобы подождать завершения процесса x миллисекунд и прервать ожидание по истечении этого времени
 - `INFINITE` – чтобы ждать завершения процесса, сколько бы времени это ни заняло

Возвращаемое значение функции ожидания

- Возвращаемым значением функции `WaitForSingleObject` может быть одно из следующих:
 - `WAIT_OBJECT_0` – ожидание прервано по причине завершения дочернего процесса (дождались)
 - `WAIT_TIMEOUT` – ожидание прервано по причине истечения таймаута (дочерний процесс не был завершён в течение этого времени)
 - `WAIT_FAILED` – ожидание завершено по причине возникновения ошибки

Ожидание нескольких процессов

- Чтобы ожидать сразу нескольких дочерних процессов, можно воспользоваться `WaitForMultipleObjects`
- Данная функция принимает массив дескрипторов тех процессов, завершения которых необходимо ждать, а также его размер
- Третьим аргументом передаётся булев флаг, означающий, что ждать нужно завершения всех процессов из переданного массива, а не одного любого
- Четвёртым аргументом передаётся значение таймаута аналогично функции `WaitForSingleObject`
- В случае завершения ожидания по причине завершения одного из процессов будет возвращено значение `WAIT_OBJECT_0` + номер завершённого процесса (индекс в массиве дескрипторов)

Пример

- В данном примере показано продолжение кода предыдущего примера, позволяющее дождаться завершения запущенного процесса

```
DWORD dwRes = WaitForSingleObject(pi.hProcess, INFINITE);

if (dwRes == WAIT_FAILED)

{

    printf("Error %x\n", GetLastError());

    ExitProcess(1);

}

printf ("Child process exited\n");
```

Получение кода завершения дочернего процесса

- Т.к. функции ожидания в Windows являются универсальными, они не позволяют получить код завершения дочернего процесса
- Для этого в Windows предназначена отдельная функция **GetExitCodeProcess**
- Функция принимает дескриптор процесса, код завершения которого необходимо получить и указатель на целочисленную переменную, по которому будет помещён код
- В случае, если процесс ещё не завершён, в переменную будет помещено значение **STILL_ACTIVE**